

B.Tech IoT Programming

Project Report

On

BATTERY HEALTH OPTIMIZATION

by

Group Leader:

**Rajvansh Malhotra (123302005) 123302005@nitkkr.ac.in
9999187079**

Other Team Members:

Satyam Kesharwani (123102033) 123102033@nitkkr.ac.in 70549 11784

Manav Singh (123302002) 123302002@nitkkr.ac.in 96435 26861

Vansh Gupta (123102022) 123102022@nitkkr.ac.in 83989 71493

Someswar Reddy(123102028) 123102028@nitkkr.ac.in 73826 39659

Faaris Quamar (123302003) 123302003@nitkkr.ac.in 76782 62528

Branch: CSE

Subject: IoT Programming (CSPC 209/ ITPC 209)

Abstract

A careful analysis of a common person's day-to-day problems which could be resolved on a scale that can assist them and ease their life has been done. The process followed a grassroots-level approach of finding the problems we could solve and diving further deeper into the day to day activities of an individual. Our observations yielded a pain point for almost every individual who owns a car, which comes out to be **battery health**. Thus, our project applies to Battery health optimization, a neat and modular **IOT** solution that provides all the information from battery percentage (real-time) to **RUL** (remaining useful life). We are extrapolating RUL by using “**Tree Ensemble Method**” like **Random Forest**^[1] which is a very powerful machine learning algorithm. This algorithm was apt for the problem statement as the data is non-linear and due to scarcity of relevant data. After producing and processing our data, we have acquired accurate results. We have achieved a commendable R^2 score of 99.89, demonstrating the effectiveness of the Random Forest model in our analysis.

1

¹ [Breiman, L. \(2001\). Random Forests. *Machine Learning*, 45\(1\), 5-32.](#)

1. *Introduction*

This project aims to fix the problems of a person who travels via car on a daily basis. A car is a basic necessity to any working individual and issues related to battery health may go unnoticed. For instance, if an individual is on their way to work and finds out that their car is unable to turn on due to degrading battery issues, it can be a disastrous situation. Data sourced from the web highlights that 51% of the world's total population uses a car daily. A normal car battery is reliable enough to last for 3-5 years and only 5-10% of these fail in between this period. After this period, almost 30-50% of the car batteries fail unexpectedly. Most of the working-class population does not worry enough about how long the car has been running due to the increased levels of stress. Unexpected failure of a car battery can induce stress on the user.

Our project corroborates insights in contexts related to battery utilisation. It offers advisories related to battery condition, allowing users to maintain awareness without significant apprehension. Random Forest has not been used in any similar project or product before, hence providing us the advantage of establishing a commendable accuracy in a short training period with scarce data. The RUL (Remaining Useful Life) helps in determining how much longer the battery can run. Predicting such data is a unique feature against current products and tools on the market.

Unique features included in this project are:

- [SOC](#) ^[2] prediction to determine RUL through [SOH](#) ^[3]
- Determination of RUL

² State of charge : [ScienceDirect](#)

³ State of health: [wikipedia](#)

2. Related Work in Project

Author	Link of published work (Research Paper/ Patent)	Year	Data	Features	Model/ Sensors/ Used	Results	Drawbacks in their Project
Khodadadi Sadabadi et al.	https://www.sciencedirect.com/science/article/abs/pii/S2352152X21001146	2021	Not Found	1)Development of an Equivalent Circuit Model (ECM) 2)Aging Studies	Voltage sensors, current sensors. Methods: Galvanostatic Current Excitation.	Development of SOH Estimation Algorithms	Complexity and Practicality Dependence on Specific Conditions
Carlos et al.	https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9036949	2020	Drawn from experiment	Uses RNN's Uses FNN's as well along with LSTM	voltage, current, etc	Gave machine learning methods to calculate soh	Didn't use tree ensemble methods, which are quick to train.
International Research Journal of Engineering and Technology (IRJET)	https://www.irjet.net/archives/V10/i5/IRJET-V10I5126.pdf	2023		Remote battery monitoring Battery performance optimization Early warning of failures	Temperature, humidity, voltage, current, gas DHT11, MQ-7, Hall effect		Requires internet connectivity Limited by sensor accuracy and data processing speed Requires initial investment in hardware and software
IoT-Based Battery Monitoring System	https://www.researchgate.net/publication/343786968_IoT-Based_Battery_Monitoring_System_for_Electric_Vehicle	2018	Monitoring Device	Monitors battery voltage Monitors vehicle location Sends notification emails	Voltage sensor SIM808 GSM/GPS/GPRS module SIM808 GSM/GPS/GPRS module	Can detect battery degradation Can track the vehicle Can alert the user to battery problems	May not be accurate May be inaccurate May be unreliable
IOT-Based Battery Management System	https://www.ripublication.com/ijaer18/ijaerv13n819.pdf	2018		Hydrogen gas detection Voltage monitoring Current monitoring Temperature monitoring Vehicle tracking	MQ-8 Hydrogen sensor Voltage divider circuit Current sensor (ACS712) or measuring voltage drop across a resistor	Detects overcharging by monitoring hydrogen emission	May not be highly accurate

Drawbacks in these existing projects

- **Khodadadi Sadabadi**

- **Complexity and Practicality:** The model uses normal algorithms and depending on time for adjustments can be too demanding for real-time, on-board diagnostics.
- **Dependence on Specific Conditions:** SOH estimations that are based on voltage are vulnerable to errors due to varying SOC of the battery, which can introduce uncertainties.

- ❖ **Carlos**

- **No clear benchmark:** The battery does not give a baseline or standard method against which the ML model is compared, making it harder to evaluate the true value of the method

- ❖ **International Research Journal of Engineering and Technology (IRJET):**

- **High Implementation Costs:** The inclusion of IoT and advanced sensors makes it costly to deploy.
- **Network Dependence:** It requires stable internet connectivity for real-time data transmission, which may be unavailable in some areas.
- **Power Draw:** Continuous monitoring can drain battery power, potentially impacting efficiency.
- **Data Security Risks:** Internet-connected systems are vulnerable to data breaches

❖ **IoT-Based Battery Monitoring System:**

- **Power Consumption:** IoT modules and sensors may draw power from the battery, reducing vehicle range.
- **Reliability in Harsh Conditions:** Extreme temperatures and vibrations may affect sensor accuracy and data transmission.

❖ **Data Security:** IoT connectivity introduces potential vulnerabilities, risking unauthorised access.

- **Cost of Implementation:** Initial installation costs may be high, impacting affordability.

Drawbacks

Weighing in the challenges and noticeable phases of the project, following drawbacks arised from the project:

1. **Cost and Complexity:** Implementing IoT components, sensors, and GPS increases the system's cost and complexity.
2. **Power Consumption:** Constant monitoring and communication can drain battery power, potentially shortening battery life.
3. **Data Reliability:** Sensor accuracy can degrade over time, affecting data quality.
4. **Connectivity Dependency:** Continuous connectivity is required for real-time data access, making it less effective in areas with poor network coverage.

3. *Project*

To implement this project, we require a set of hardware components and software. Using sensors (current, voltage, humidity, temperature) and Arduino.

We initiate our process by gathering critical parameters including current, voltage, humidity, and temperature through their respective sensors. Following this data acquisition, we then move forward by applying a controlled load on the battery to dissipate it. We constantly monitor for a few hours in different conditions to ensure robustness in our model and to mimic different environmental conditions, thereby addressing potential data imbalances so our model isn't biased to one climate.

We then add all the recorded data to a CSV file which is then further fed to our ML model.(Random Forest) We get predictions of SOC from which SOH is predicted, and thus RUL is predicted.

Sample data:

current	voltage	percentage	soc	temp_new	discharge rate	soh	dod
3.041	12.15	44.02	82.5	30.94	0.4344285714	96.42857143	2.897638571
3.041	12.14	42.89	82	30.99	0.4344285714	96.58730159	2.897638571
3.025	12.15	44.02	82.5	30.89	0.4321428571	96.58730159	2.882392857
2.977	12.17	46.84	83.5	30.89	0.4252857143	96.58730159	2.836655714

The flowchart below (on page number 8) represents a smart Battery Management System (BMS) that combines IoT devices and machine learning to continuously monitor and predict battery health. The system begins with sensors attached to the battery, which measure essential metrics like current, voltage, temperature, and humidity. This data helps assess the battery's condition in real time. If any sensor fails or is not properly connected, the system detects the issue and prompts a recheck, ensuring reliable data collection.

Once the sensors gather data, it's organised and saved in a CSV file for easier processing. The recent data is then transmitted wirelessly through an ESP32 module, a microcontroller equipped with Wi-Fi and Bluetooth, which connects the data to the Arduino Cloud. In the cloud, the system's machine learning model—trained on historical sensor data—is stored and ready to make predictions. This model is saved as a .pkl file, containing all the learned patterns required for accurate predictions.

The cloud-based machine learning model processes new incoming data, identifying patterns that might signal battery performance issues or possible failures. These predictions, along with other displayable data, are sent to a user-friendly app, where users can easily monitor battery health and receive alerts about potential problems. This system allows for real-time tracking and proactive maintenance, making it particularly useful in applications where battery reliability is crucial, such as in electric vehicles or renewable energy storage. By combining IoT and machine learning, this BMS enables smarter, more efficient battery management and provides users with valuable insights to extend battery life and prevent unexpected failures.

The project's novelty lies in its cost-effective design, allowing for successful implementation with minimal investment. This approach not only demonstrates the feasibility of the project but also highlights its accessibility, making it possible for individuals or organisations with limited resources to engage in similar initiatives.

The same is explained through the following diagrams:

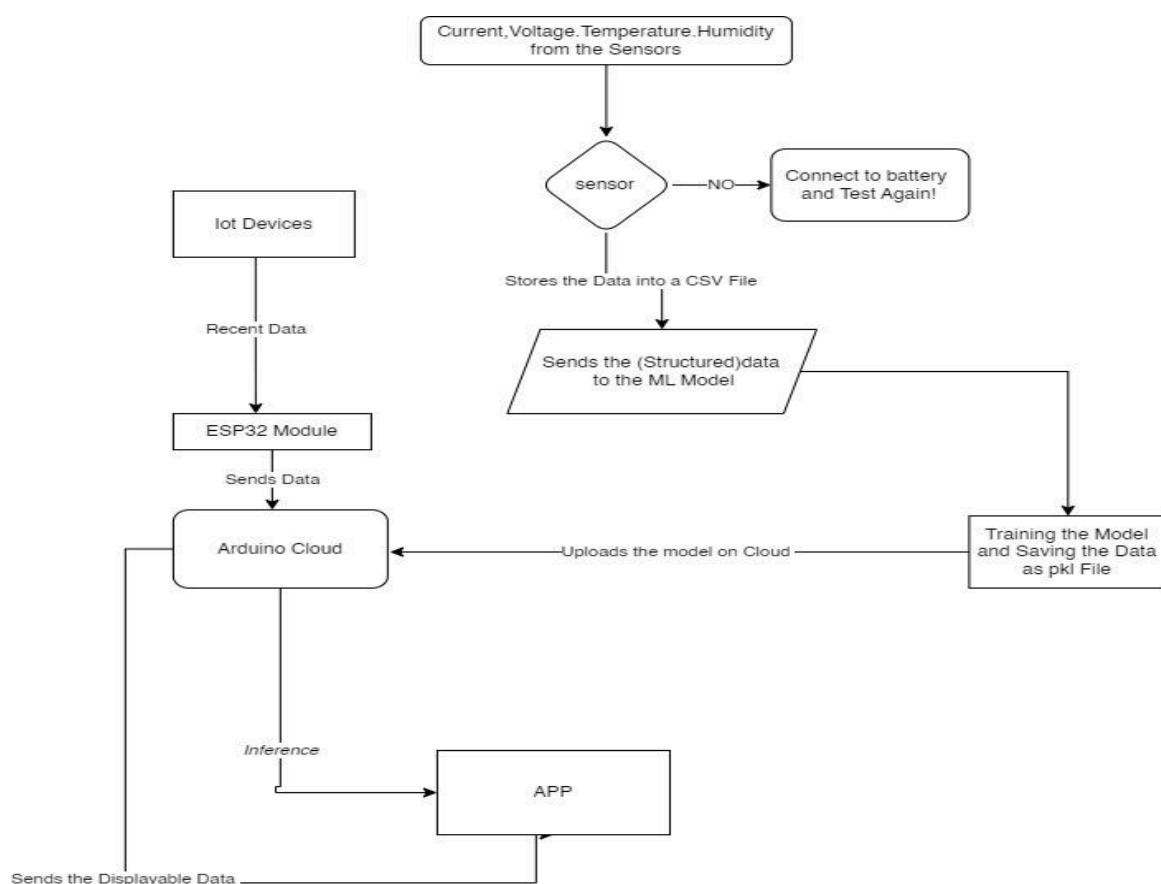
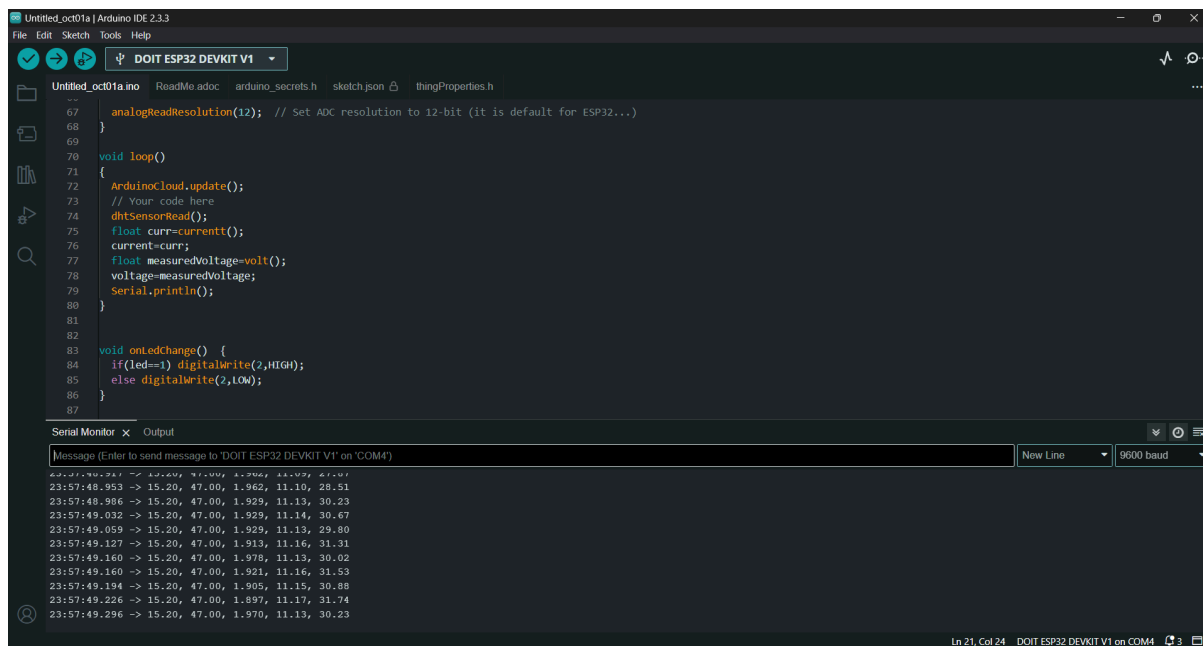


Figure 1: BMS Flow Diagram
(Depicting the Flow of Data and Predictions made by the ML Model)

Working Project



Untitled_0cd01a | Arduino IDE 2.3.3

File Edit Sketch Tools Help

DOIT ESP32 DEVKIT V1

Untitled_0cd01a.ino | README.adoc | arduino_secrets.h | sketch.json | thingProperties.h

```
67 analogReadResolution(12); // Set ADC resolution to 12-bit (it is default for ESP32...)
68 }
69
70 void loop()
71 {
72   ArduinoCloud.update();
73   // Your code here
74   dhtSensorRead();
75   float curr=currentt();
76   current=curr;
77   float measuredVoltage=volt();
78   voltage=measuredVoltage;
79   Serial.println();
80 }
81
82
83 void onLedChange() {
84   if(led==1) digitalWrite(2,HIGH);
85   else digitalWrite(2,LOW);
86 }
87
```

Serial Monitor x Output

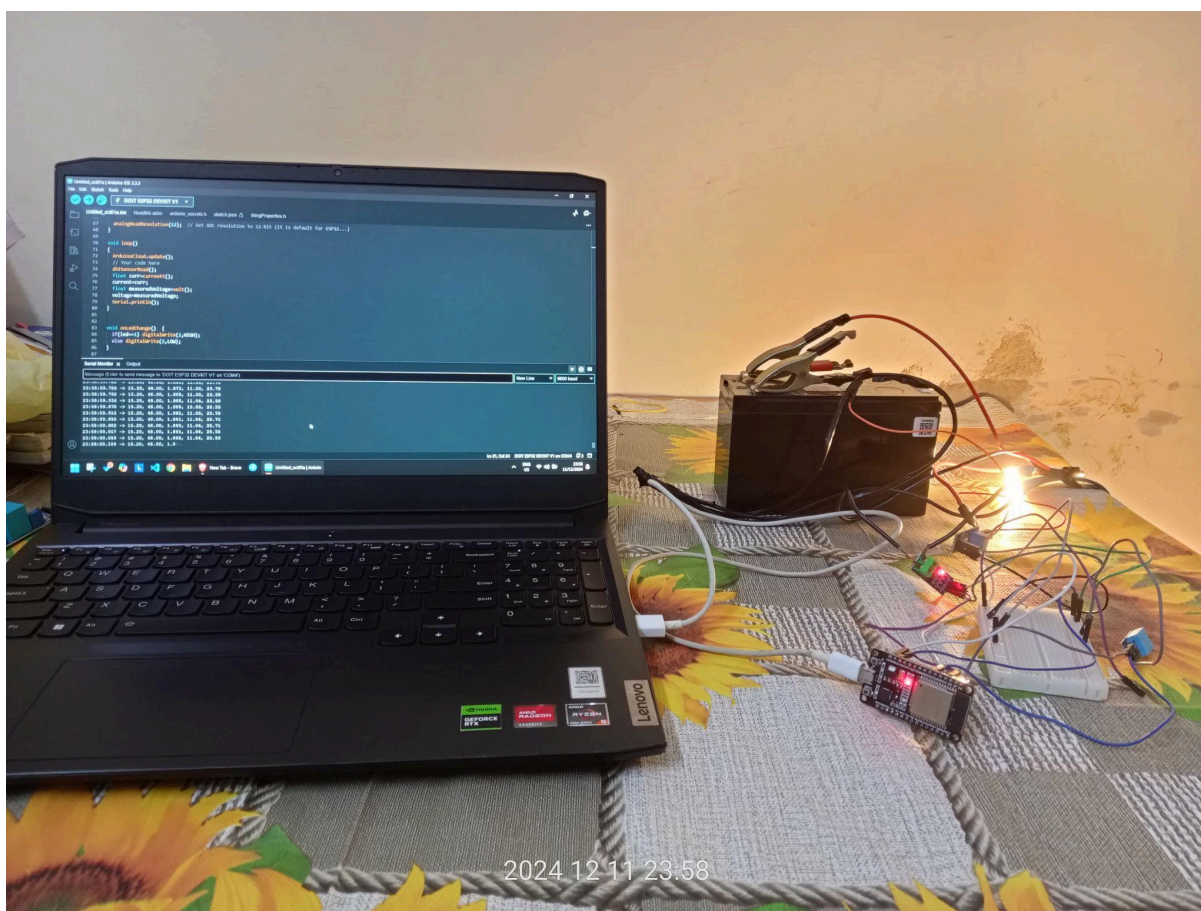
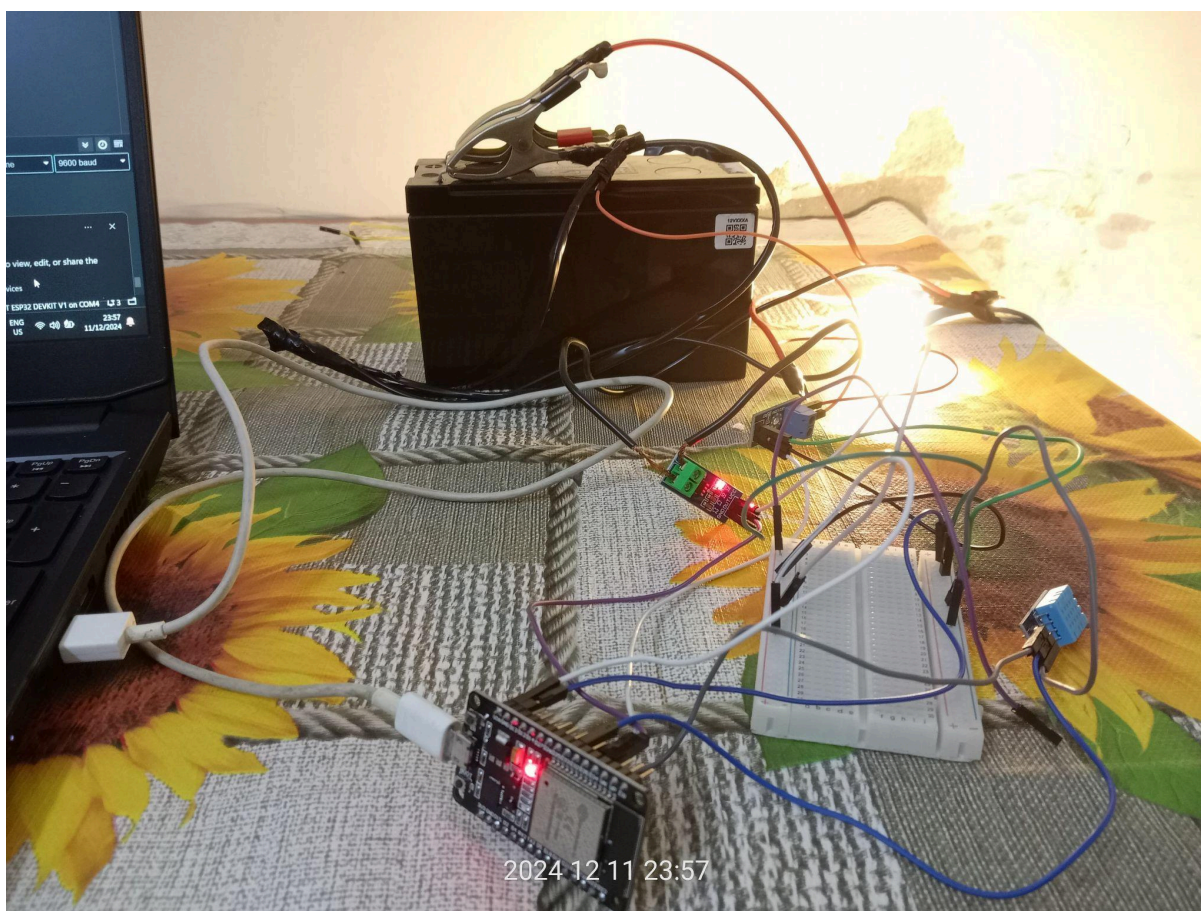
Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM4')

New Line 9600 baud

```
23:57:48.953 -> 15.20, 47.00, 1.962, 11.10, 28.51
23:57:48.986 -> 15.20, 47.00, 1.929, 11.13, 30.23
23:57:49.032 -> 15.20, 47.00, 1.929, 11.14, 30.67
23:57:49.059 -> 15.20, 47.00, 1.929, 11.13, 29.80
23:57:49.127 -> 15.20, 47.00, 1.913, 11.16, 31.31
23:57:49.160 -> 15.20, 47.00, 1.978, 11.13, 30.02
23:57:49.160 -> 15.20, 47.00, 1.921, 11.16, 31.53
23:57:49.194 -> 15.20, 47.00, 1.905, 11.15, 30.88
23:57:49.226 -> 15.20, 47.00, 1.897, 11.17, 31.74
23:57:49.296 -> 15.20, 47.00, 1.970, 11.13, 30.23
```

Ln 21, Col 24 DOIT ESP32 DEVKIT V1 on COM4





XGBoost vs. Random Forest

One of the vital elements of our project was the choice of Random Forest over XGBoost. While there was no significant difference in accuracy between the two models, a key distinction was the training time—Random Forest excelled in this metric compared to XGBoost.

However, upon examining both algorithms comparatively, it became clear that Random Forest performs efficiently on small to medium datasets but faces challenges with larger datasets due to its significant memory demands. Given the current scope and scale of the project, Random Forest was the most suitable choice.

Random Forest:

```
# cross val score along with the r2 score for random forest
cv_scores = cross_val_score(rf, x_train, y_train, cv=5, scoring='r2')
print("Mean cross-validation score:", cv_scores.mean())
rf= RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(x_train, y_train)
y_pred = rf.predict(x_test)

r2_rf = r2_score(y_test, y_pred)
print("The r2 score for random forest is:", r2_rf)
```

```
Mean cross-validation score: 0.9992266481625791
The r2 score for random forest is: 0.9989397226944803
```

Xgboost :

```
# cross val score along with the r2 score for xgboost is :
xgb_reg = xgb.XGBRegressor()

cv_scores = cross_val_score(xgb_reg, x_train, y_train, cv=5, scoring='r2')
print("Mean cross-validation score:", cv_scores.mean())

xgb_reg.fit(x_train, y_train)
y_pred = xgb_reg.predict(x_test)
test_r2_score = r2_score(y_test, y_pred)
print("Test R^2 Score:", test_r2_score)
```

```
Mean cross-validation score: 0.9985234859438853
Test R^2 Score: 0.9989555635564643
```

4. *Conclusion*

This project applies to a problem faced by millions of car owners around the world; the unexpected failure of car batteries. Broadly translated, our findings indicate that battery health is a pain point for car owners worldwide. Due to the reliability of cars for daily commutes, this solution aims to resolve the risks associated with sudden breakdowns.

To solve this issue, we have developed a user-friendly IoT solution that provides the RUL (Remaining Useful Life) of a car battery. Using machine learning algorithms like Random Forest and XGBoost, it is possible to predict battery health with a high degree of accuracy. These algorithms were best suited for this task because of the complex variance in battery degradation patterns. The use of Tree Ensemble methods led to building our own dataset which was essential for training the machine learning models. Random

This solution provides the user with a notification which displays the battery information including charge levels and provides predictions as to when the battery is likely to fail. The RUL (Remaining Usage Life) estimation is a feature that helps users plan ahead and prevent failures in the battery which could disrupt daily routines. This system reduces the need for stress over the battery's performance and notifies the users to take preventive measures when necessary.

Overall, this project serves as a tool to enhance the reliability and longevity of car batteries. This project underlines the importance of making use of IoT technology and machine

learning to solve everyday problems. Future developments in car maintenance systems will benefit from it. With additional work, this technology may be able to provide repair options for other automotive sector components in addition to car batteries. In conclusion, our study is a major step toward better user comfort and data-driven, intelligent battery management and optimization.

5. *Future Works*

In the future, this Battery Health Optimization project could expand in several meaningful ways to enhance its usability, accuracy, and impact.

One potential direction is securing a dedicated cloud subscription to handle large-scale data storage and processing. This would allow for real-time monitoring and data analytics for a wider user base, scaling the system for commercial or enterprise applications. Partnerships with battery manufacturers represent another strategic expansion. By integrating the monitoring system directly into batteries during production, data can be gathered continuously from different battery types and usage conditions, improving the model's predictive accuracy and applicability. Such partnerships would not only expand the data pool but also promote this system as a standard in new-generation batteries.

Investing in more precise sensors is another area for future development. Currently, the system relies on cost-effective sensors, which are sufficient for a prototype, but high-precision sensors would yield more reliable and nuanced data, especially under variable conditions. With higher accuracy, predictions of RUL and SOC would be refined, enhancing the reliability of alerts and user notifications.

Finally, the project could incorporate integrations with platforms like ThingSpeak, an IoT analytics service, to enable better data visualisation and analytics. This addition would allow users to access their battery health data remotely, monitor trends over time, and receive predictive maintenance insights.

References

1. Sanjana Gajbhiye (2024, March). 51% of all humans rely on car commutes every day.
2. Research papers: ieeexplore.ieee.org , google scholar

